Tutorial: Tài liệu hóa REST API với Spring Boot 4.1 + springdoc-openapi 3 (OpenAPI 3.1)

Oct 16, 2026 · @Ng Tuân

1. Tổng quan

Tutorial này xây dựng REST API quản lý sản phẩm (`Product` , CRUD) trên MySQL với Spring Boot 4.1 và JDK 25, rồi tài liệu hóa API bằng springdoc-openapi 3.1: đặc tả OpenAPI 3.1 và Swagger UI được sinh tự động từ code.

Các kỹ thuật được áp dụng:

- DTO viết bằng Java `record` , ràng buộc Bean Validation tự hiện lên spec (`required` , `maxLength` , `minimum`).
- Annotation tài liệu tách ra interface `ProductApi` , controller chỉ chứa logic.
- Lỗi trả về theo chuẩn RFC 9457 (`ProblemDetail` , `application/problem+json`) và được mô tả trong spec.
- MySQL tự khởi động qua Docker Compose support của Spring Boot, không cần sửa `application.properties` khi đổi môi trường.
- Test tự động kiểm tra spec, tránh tài liệu "trôi" khỏi code.

Kết quả cuối cùng: Swagger UI tại `http://localhost:8080/swagger-ui.html` , spec JSON tại `/v3/api-docs` , YAML tại `/v3/api-docs.yaml` .

2. Yêu cầu môi trường

Thành phần	Phiên bản	Ghi chú
JDK	25 (LTS)	Spring Boot 4.1 yêu cầu tối thiểu Java 17
Spring Boot	4.1.1	Nền Spring Framework 7, Jakarta EE 11, Jackson 3
springdoc-openapi	3.1.1	Dòng 3.x dành cho Spring Boot 4
Chuẩn đặc tả	OpenAPI 3.1	
MapStruct	1.6.3	

Thành phần	Phiên bản	Ghi chú
MySQL	8.4 LTS	Chạy trong container Docker
Maven	3.9+	Dùng Maven Wrapper đi kèm project
Docker	Docker Desktop hoặc Docker Engine + Compose v2	Để Spring Boot tự khởi động MySQL

Patch version thay đổi hằng tháng: khi tạo project, lấy bản patch mới nhất của 4.1.x trên start.spring.io và bản 3.1.x mới nhất của springdoc.

3. Khởi tạo project

Tạo project trên start.spring.io với: Maven, Java 25, Spring Boot 4.1.x, group `com.fptaptech`, artifact `catalog`. Chọn dependency: Spring Web, Spring Data JPA, Validation, MySQL Driver, Docker Compose Support, H2 (cho test). springdoc và MapStruct thêm tay ở bước 4.

Cấu trúc thư mục đích (đóng gói theo tính năng):

```
catalog/
├── compose.yaml                    # MySQL cho môi trường dev
├── Dockerfile
├── mvnw, mvnw.cmd, .mvn/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/com/fptaptech/catalog/
    │   │   ├── CatalogApplication.java
    │   │   ├── config/
    │   │   │   └── OpenApiConfig.java
    │   │   ├── common/
    │   │   │   └── GlobalExceptionHandler.java
    │   │   └── product/
    │   │       ├── Product.java                # JPA entity
    │   │       ├── ProductRequest.java         # DTO vào (record)
    │   │       ├── ProductResponse.java        # DTO ra (record)
    │   │       ├── ProductMapper.java          # MapStruct
    │   │       ├── ProductRepository.java
    │   │       ├── ProductService.java
    │   │       ├── ProductNotFoundException.java
    │   │       ├── ProductApi.java             # hợp đồng API + annotation
OpenAPI
    │   │       └── ProductController.java      # implements ProductApi
```

```
    |   └── resources/
    |       ├── application.yaml
    |       └── application-prod.yaml
    └── test/java/com/fptaptech/catalog/
        └── OpenApiSpecTest.java
```

4. pom.xml

Dependency `springdoc-openapi-starter-webmvc-ui` sinh đặc tả OpenAPI và Swagger UI. Spring Boot 4 đã module hóa starter: dùng `spring-boot-starter-webmvc` cho Spring MVC.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>

    <groupId>com.fptaptech</groupId>
    <artifactId>catalog</artifactId>
    <version>1.0.0</version>
    <name>catalog</name>

    <properties>
        <java.version>25</java.version>
        <springdoc.version>3.1.1</springdoc.version>
        <mapstruct.version>1.6.3</mapstruct.version>
    </properties>

    <dependencies>
        <!-- Web, JPA, Validation -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- OpenAPI 3.1 + Swagger UI -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>${springdoc.version}</version>
        </dependency>

        <!-- MapStruct -->
        <dependency>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct</artifactId>
            <version>${mapstruct.version}</version>
        </dependency>

        <!-- Database -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-docker-compose</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!-- Test -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <!-- JDK 23+ tắt annotation processing ngầm định: phải
khai báo processor tường minh -->
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.mapstruct</groupId>
                            <artifactId>mapstruct-processor</artifactId>
                            <version>${mapstruct.version}</version>
                        </path>
                    </annotationProcessorPaths>
                    <compilerArgs>
                        <arg>-Amapstruct.defaultComponentModel=spring</arg>
                    </compilerArgs>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

5. Code domain Product

Mọi annotation OpenAPI nằm ở DTO và interface `ProductApi`; Entity, Service, Controller không phụ thuộc thư viện tài liệu.

5.1 Entity

```
package com.fptaptech.catalog.product;

import jakarta.persistence.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.math.BigDecimal;
import java.time.Instant;
```

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String name;

    @Column(length = 500)
    private String description;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal price;

    @CreationTimestamp
    @Column(updatable = false)
    private Instant createdAt;

    @UpdateTimestamp
    private Instant updatedAt;

    // getters & setters (MapStruct cần setter để map vào entity)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt;
}
    public Instant getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(Instant updatedAt) { this.updatedAt = updatedAt;
}
}
```

5.2 DTO dạng record + `@Schema`

`@NotBlank`, `@NotNull`, `@Size`, `@DecimalMin` được springdoc đọc và chuyển thành `required`, `maxLength`, `minimum` trong spec — không cần khai báo lặp trong `@Schema`.

```
package com.fptaptech.catalog.product;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.*;

import java.math.BigDecimal;

@Schema(description = "Dữ liệu tạo hoặc cập nhật sản phẩm")
public record ProductRequest(

        @Schema(description = "Tên sản phẩm", example = "Bàn phím cơ K8 Pro")
        @NotBlank @Size(max = 120)
        String name,

        @Schema(description = "Mô tả ngắn", example = "Switch Gateron Brown,
layout 87 phím")
        @Size(max = 500)
        String description,

        @Schema(description = "Giá bán (VND)", example = "1590000")
        @NotNull @DecimalMin("0.0") @Digits(integer = 10, fraction = 2)
        BigDecimal price
) {}
```

```
package com.fptaptech.catalog.product;

import io.swagger.v3.oas.annotations.media.Schema;

import java.math.BigDecimal;
import java.time.Instant;

@Schema(description = "Thông tin sản phẩm trả về")
public record ProductResponse(
        @Schema(example = "1") Long id,
        @Schema(example = "Bàn phím cơ K8 Pro") String name,
        String description,
        @Schema(example = "1590000") BigDecimal price,
        Instant createdAt,
        Instant updatedAt
) {}
```

5.3 Mapper, Repository, Exception

```
package com.fptaptech.catalog.product;

import org.mapstruct.*;

import java.util.List;

@Mapper(componentModel = MappingConstants.ComponentModel.SPRING)
public interface ProductMapper {

    ProductResponse toResponse(Product product);

    List<ProductResponse> toResponses(List<Product> products);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "updatedAt", ignore = true)
    Product toEntity(ProductRequest request);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "updatedAt", ignore = true)
    void updateEntity(ProductRequest request, @MappingTarget Product
product);
}
```

```
package com.fptaptech.catalog.product;

import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {}
```

```
package com.fptaptech.catalog.product;

public class ProductNotFoundException extends RuntimeException {
    private final Long id;

    public ProductNotFoundException(Long id) {
        super("Không tìm thấy sản phẩm id=" + id);
        this.id = id;
    }

    public Long getId() { return id; }
}
```

5.4 Service

```
package com.fptaptech.catalog.product;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional(readOnly = true)
public class ProductService {

    private final ProductRepository repository;
    private final ProductMapper mapper;

    public ProductService(ProductRepository repository, ProductMapper mapper)
{
        this.repository = repository;
        this.mapper = mapper;
    }

    public List<ProductResponse> findAll() {
        return mapper.toResponses(repository.findAll());
    }

    public ProductResponse findById(Long id) {
        return mapper.toResponse(getOrThrow(id));
    }

    @Transactional
    public ProductResponse create(ProductRequest request) {
        Product saved = repository.save(mapper.toEntity(request));
        return mapper.toResponse(saved);
    }

    @Transactional
    public ProductResponse update(Long id, ProductRequest request) {
        Product product = getOrThrow(id);
        mapper.updateEntity(request, product);
        // flush để @UpdateTimestamp được ghi trước khi map ra response
        return mapper.toResponse(repository.saveAndFlush(product));
    }

    @Transactional
    public void delete(Long id) {
```

```
        repository.delete(getOrThrow(id));
    }

    private Product getOrThrow(Long id) {
        return repository.findById(id).orElseThrow(() -> new
ProductNotFoundException(id));
    }
}
```

5.5 Hợp đồng API: `ProductApi`

Interface giữ mapping + toàn bộ annotation tài liệu. Spring MVC đọc được `@GetMapping`, `@PathVariable`, `@Valid` khai báo trên interface, nên controller rất gọn.

```
package com.fptaptech.catalog.product;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.headers.Header;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@Tag(name = "Products", description = "Quản lý danh mục sản phẩm")
@RequestMapping("/api/v1/products")
public interface ProductApi {

    @Operation(summary = "Lấy danh sách sản phẩm")
    @ApiResponse(responseCode = "200", description = "Danh sách sản phẩm (có
thể rỗng)")
    @GetMapping
    List<ProductResponse> findAll();

    @Operation(summary = "Lấy chi tiết sản phẩm")
    @ApiResponse(responseCode = "200", description = "Tìm thấy sản phẩm")
    @ApiResponse(responseCode = "404", description = "Sản phẩm không tồn
tại",
            content = @Content(mediaType = "application/problem+json",
```

```
                    schema = @Schema(implementation = ProblemDetail.class)))
    @GetMapping("/{id}")
    ProductResponse findById(
            @Parameter(description = "ID sản phẩm", example = "1")
@PathVariable Long id);

    @Operation(summary = "Tạo sản phẩm mới")
    @ApiResponse(responseCode = "201", description = "Đã tạo",
            headers = @Header(name = "Location", description = "URI của sản
phẩm vừa tạo"))
    @ApiResponse(responseCode = "400", description = "Dữ liệu không hợp lệ",
            content = @Content(mediaType = "application/problem+json",
                    schema = @Schema(implementation = ProblemDetail.class)))
    @PostMapping
    ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest
request);

    @Operation(summary = "Cập nhật toàn bộ sản phẩm")
    @ApiResponse(responseCode = "200", description = "Đã cập nhật")
    @ApiResponse(responseCode = "400", description = "Dữ liệu không hợp lệ",
            content = @Content(mediaType = "application/problem+json",
                    schema = @Schema(implementation = ProblemDetail.class)))
    @ApiResponse(responseCode = "404", description = "Sản phẩm không tồn
tại",
            content = @Content(mediaType = "application/problem+json",
                    schema = @Schema(implementation = ProblemDetail.class)))
    @PutMapping("/{id}")
    ProductResponse update(@Parameter(description = "ID sản phẩm", example =
"1") @PathVariable Long id,
                           @Valid @RequestBody ProductRequest request);

    @Operation(summary = "Xóa sản phẩm")
    @ApiResponse(responseCode = "204", description = "Đã xóa")
    @ApiResponse(responseCode = "404", description = "Sản phẩm không tồn
tại",
            content = @Content(mediaType = "application/problem+json",
                    schema = @Schema(implementation = ProblemDetail.class)))
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    void delete(@Parameter(description = "ID sản phẩm", example = "1")
@PathVariable Long id);
}
```

5.6 Controller

```
package com.fptaptech.catalog.product;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

@RestController
public class ProductController implements ProductApi {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @Override
    public List<ProductResponse> findAll() {
        return service.findAll();
    }

    @Override
    public ProductResponse findById(Long id) {
        return service.findById(id);
    }

    @Override
    public ResponseEntity<ProductResponse> create(ProductRequest request) {
        ProductResponse created = service.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @Override
    public ProductResponse update(Long id, ProductRequest request) {
        return service.update(id, request);
    }

    @Override
```

```
    public void delete(Long id) {
        service.delete(id);
    }
}
```

6. Cấu hình OpenAPI

springdoc tự quét controller để sinh spec; bean `OpenAPI` chỉ dùng để khai báo metadata (title, version, contact, server).

6.1 Bean metadata

```
package com.fptaptech.catalog.config;

import io.swagger.v3.oas.models.ExternalDocumentation;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    OpenAPI catalogOpenApi(@Value("${app.api.version:1.0.0}") String version,
                           @Value("${app.api.server-
url:http://localhost:8080}") String serverUrl) {
        return new OpenAPI()
                .info(new Info()
                        .title("Catalog API")
                        .version(version)
                        .description("REST API quản lý danh mục sản phẩm")
                        .contact(new Contact().name("Catalog
Team").email("catalog@example.com"))
                        .license(new License().name("Apache 2.0")

.url("https://www.apache.org/licenses/LICENSE-2.0")))
                .servers(List.of(new Server().url(serverUrl).description("Môi
trường hiện tại")))
```

```
                .externalDocs(new ExternalDocumentation()
                        .description("Hướng dẫn tích hợp")
                        .url("https://example.com/docs/catalog"));
    }
}
```

6.2 application.yaml

```
spring:
  application:
    name: catalog
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: update          # tutorial; dự án thật dùng Flyway/Liquibase
  mvc:
    problemdetails:
      enabled: true             # lỗi framework (400, 405, 415...) trả về
ProblemDetail

app:
  api:
    version: 1.0.0

springdoc:
  api-docs:
    path: /v3/api-docs
    version: openapi_3_1
  swagger-ui:
    path: /swagger-ui.html      # redirect tới /swagger-ui/index.html
    operations-sorter: method
    tags-sorter: alpha
    display-request-duration: true
    try-it-out-enabled: true
  default-produces-media-type: application/json
  paths-to-match: /api/**
```

Không có `spring.datasource.*`: Docker Compose support tự sinh connection details từ `compose.yaml` (mục 8).

6.3 Tắt tài liệu trên production

`src/main/resources/application-prod.yaml`:

```
springdoc:
  api-docs:
    enabled: false
  swagger-ui:
    enabled: false
```

Nếu cần công khai spec trên production, giữ `api-docs` nhưng đặt sau Spring Security, chỉ mở cho role nội bộ.

6.4 Annotation hay dùng

Annotation	Đặt ở	Tác dụng
`@Tag`	Interface / controller	Nhóm endpoint trên Swagger UI
`@Operation`	Method	`summary`, `description`, `operationId`, `deprecated`
`@ApiResponse`	Method	Mô tả từng mã HTTP, kiểu nội dung, header trả về
`@Parameter`	Tham số	Mô tả, ví dụ, `required`, `hidden`
`@Schema`	DTO / field	Mô tả, `example`, `accessMode`, `allowableValues`
`@Hidden`	Method / class	Ẩn khỏi spec
`@ParameterObject`	Tham số `Pageable` / object query	Bung từng field thành query param

7. Xử lý lỗi chuẩn RFC 9457

Mọi lỗi trả về `application/problem+json` với cấu trúc `type`, `title`, `status`, `detail`, `instance` cộng các trường mở rộng; spec mô tả đúng cấu trúc đó nhờ `@Schema(implementation = ProblemDetail.class)` ở mục 5.5.

```
package com.fptaptech.catalog.common;

import com.fptaptech.catalog.product.ProductNotFoundException;
import org.springframework.http.*;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
```

```
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityException
Handler;

import java.net.URI;
import java.time.Instant;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    private static final String ERROR_BASE =
"https://api.example.com/errors/";

    @ExceptionHandler(ProductNotFoundException.class)
    ProblemDetail handleNotFound(ProductNotFoundException ex) {
        ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setType(URI.create(ERROR_BASE + "product-not-found"));
        pd.setTitle("Product not found");
        pd.setProperty("productId", ex.getId());
        pd.setProperty("timestamp", Instant.now());
        return pd;
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        ProblemDetail pd = ex.getBody();
        pd.setType(URI.create(ERROR_BASE + "validation"));
        pd.setTitle("Validation failed");
        pd.setProperty("timestamp", Instant.now());
        pd.setProperty("errors",
ex.getBindingResult().getFieldErrors().stream()
                .map(fe -> Map.of("field", fe.getField(),
                                  "message",
String.valueOf(fe.getDefaultMessage())))
                .toList());
        return handleExceptionInternal(ex, pd, headers, status, request);
    }
}
```

Ví dụ phản hồi khi `POST /api/v1/products` với `name` rỗng:

```
{
  "type": "https://api.example.com/errors/validation",
  "title": "Validation failed",
  "status": 400,
  "detail": "Invalid request content.",
  "instance": "/api/v1/products",
  "timestamp": "2026-10-16T03:12:45.120Z",
  "errors": [
    { "field": "name", "message": "must not be blank" }
  ]
}
```

springdoc cũng quét `@RestControllerAdvice` để sinh response chung. Nếu thấy các mã lỗi bị gắn vào mọi endpoint ngoài ý muốn, đặt `springdoc.override-with-generic-response: false` và giữ `@ApiResponse` khai báo tường minh trong `ProductApi`.

8. Chạy ứng dụng

Cách nhanh nhất: `./mvnw spring-boot:run` — Spring Boot đọc `compose.yaml`, tự khởi động MySQL, chờ container sẵn sàng rồi cấu hình datasource.

8.1 compose.yaml (dev)

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: catalog
      MYSQL_USER: catalog
      MYSQL_PASSWORD: catalog_pw
      MYSQL_ROOT_PASSWORD: root_pw
    ports:
      - "3306"          # cổng ngẫu nhiên trên host, Boot tự dò
```

```
./mvnw spring-boot:run
# Log sẽ có: "Using Docker Compose file .../compose.yaml" và "Started
CatalogApplication"
```

8.2 Chạy với MySQL có sẵn (không Docker)

```
./mvnw spring-boot:run \
  -Dspring-boot.run.arguments="--spring.docker.compose.enabled=false \
  --spring.datasource.url=jdbc:mysql://localhost:3306/catalog \
  --spring.datasource.username=catalog --
spring.datasource.password=catalog_pw"
```

8.3 Đóng gói image

Cách 1, không cần Dockerfile: `./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=catalog:1.0.0` (Cloud Native Buildpacks).

Cách 2, Dockerfile multi-stage:

```
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY . .
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=build /src/target/catalog-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar", "--spring.profiles.active=prod"]
```

Khi chạy image với profile `prod`, truyền `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD` qua biến môi trường; Swagger UI bị tắt theo mục 6.3.

9. Kiểm thử

9.1 Trên Swagger UI

1. Mở `http://localhost:8080/swagger-ui.html` — trang chuyển tới `/swagger-ui/index.html`, hiện tag **Products** với 5 operation.
2. Mở `POST /api/v1/products` → **Try it out** → body mẫu đã điền sẵn từ `example` của `@Schema` → **Execute**. Kỳ vọng: `201`, header `Location: http://localhost:8080/api/v1/products/1`.
3. Gọi lại với `"name": ""` → `400` dạng `application/problem+json` như mục 7.
4. `GET /api/v1/products` → `200` và mảng sản phẩm; `GET /api/v1/products/999` → `404`.

5. Mở mục **Schemas** cuối trang: `ProductRequest` có `required: [name, price]`, `name.maxLength = 120`.

9.2 Bằng curl

```
curl -i -X POST http://localhost:8080/api/v1/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"Bàn phím cơ K8 Pro","description":"Gateron
Brown","price":1590000}'

curl -s http://localhost:8080/api/v1/products | jq
curl -s http://localhost:8080/v3/api-docs.yaml -o openapi.yaml   # xuất spec
để chia sẻ cho FE/đối tác
```

9.3 Test tự động cho spec

Test này fail nếu ai đó vô tình đổi đường dẫn, xóa mô tả lỗi 404 hay bỏ ràng buộc `required`. Trong test, Docker Compose bị bỏ qua mặc định và H2 (scope test) được dùng làm DB nhúng.

```
package com.fptaptech.catalog;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc; //
package mới ở Boot 4
import org.springframework.test.web.servlet.MockMvc;

import static org.hamcrest.Matchers.*;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
class OpenApiSpecTest {

    @Autowired
    MockMvc mvc;

    @Test
    void specDescribesProductApi() throws Exception {
```

```
        mvc.perform(get("/v3/api-docs"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.openapi", startsWith("3.1")))
           .andExpect(jsonPath("$.info.title").value("Catalog API"))

.andExpect(jsonPath("$.paths['/api/v1/products'].post.responses['201']").exis
ts())

.andExpect(jsonPath("$.paths['/api/v1/products/{id}'].get.responses['404']").
exists())

.andExpect(jsonPath("$.components.schemas.ProductRequest.required",
                  containsInAnyOrder("name", "price")));
    }
}
```

Chạy: `./mvnw test`. Nếu IDE báo không tìm thấy `AutoConfigureMockMvc`, kiểm tra đã có `spring-boot-starter-webmvc-test` trong pom.

10. Bài tập mở rộng: Order Service cho TechShop

Xây dựng dịch vụ đặt hàng dùng lại module `product` ở trên, trong đó spec OpenAPI là **hợp đồng** cho 3 nhóm client: app khách hàng, trang quản trị nội bộ và đối tác vận chuyển. Frontend và đối tác sẽ code theo spec trước khi backend xong, nên spec phải đủ chính xác để sinh client tự động.

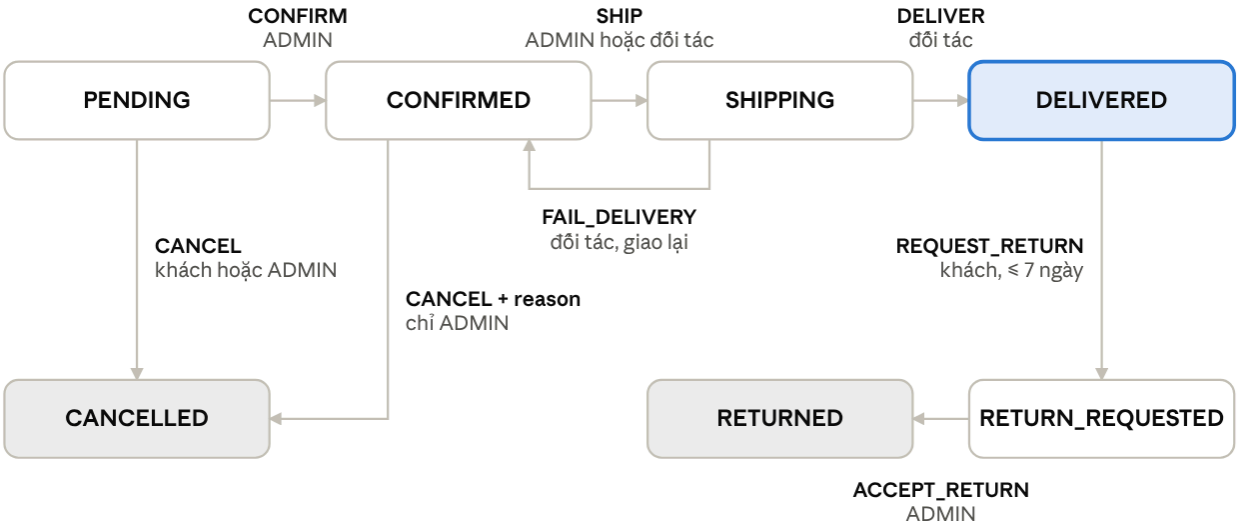
10.1 Mô hình dữ liệu

Thực thể	Trường chính	Ghi chú
`Customer`	id, fullName, email, phone	Email duy nhất
`Order`	id, code (`ORD-2026-000123`), customerId, status, shippingAddress, totalAmount, currency, version, createdAt	`version` dùng optimistic locking và làm ETag
`OrderItem`	productId, productName, unitPrice, quantity, lineTotal	`productName`, `unitPrice` là bản chụp tại thời điểm đặt
`Payment` (đa hình)	`method` = `COD` | `CARD` | `BANK_TRANSFER`	CARD: provider, last4; BANK_TRANSFER: bankCode, transferRef; COD: không có trường riêng

Thực thể	Trường chính	Ghi chú
`OrderEvent`	orderId, fromStatus, toStatus, action, actor, reason, at	Nhật ký chuyển trạng thái

10.2 Máy trạng thái

Đơn chỉ hủy được trước khi giao; trả hàng trong 7 ngày sau khi giao



Ô nền xám: trạng thái kết thúc. Mỗi mũi tên là một giá trị action, kèm vai trò được phép gọi.

máy trạng thái đơn hàng · 7 trạng thái, 8 hành động

Hành động của ADMIN và khách đi qua `POST /api/orders/{code}/transitions`; hành động của đối tác đi qua webhook `/api/webhooks/shipping`. Spec phải liệt kê `action` dạng enum và mô tả vai trò được phép cho từng giá trị.

Chuyển trạng thái không hợp lệ trả `409 Conflict` với mã lỗi `ORDER_INVALID_TRANSITION`.

10.3 Danh sách endpoint

Method	Path	Nhóm spec	Xác thực	Mô tả
POST	`/api/orders`	public	Bearer JWT (CUSTOMER)	Tạo đơn; header `Idempotency-Key` bắt buộc

Method	Path	Nhóm spec	Xác thực	Mô tả
GET	`/api/orders`	public	Bearer JWT	Phân trang + lọc `status`, `from`, `to`, `minTotal`; sort `createdAt`, `totalAmount`. CUSTOMER chỉ thấy đơn của mình
GET	`/api/orders/{code}`	public v1 + v2	Bearer JWT	v1 trả `totalAmount` dạng số; v2 trả `total: {value, currency}`. v1 đánh dấu deprecated
POST	`/api/orders/{code}/transitions`	admin + public	Bearer JWT (ADMIN mọi action; CUSTOMER chỉ CANCEL, REQUEST_RETURN trên đơn của mình)	Body `{action, reason}`; header `If-Match` bắt buộc
POST	`/api/orders/{code}/invoice`	admin	Bearer JWT (ADMIN)	Upload `multipart/form-data`, chỉ PDF, ≤ 5 MB
GET	`/api/orders/{code}/invoice`	public	Bearer JWT	Trả `application/pdf`
POST	`/api/webhooks/shipping`	partner	API key header `X-Partner-Key`	Đối tác báo `SHIPPING` / `DELIVERED` / giao thất bại
GET	`/api/admin/reports/daily-revenue`	admin	Bearer JWT (ADMIN)	Doanh thu theo ngày, lọc `from` / `to`

10.4 Quy tắc nghiệp vụ

- Client không được gửi giá: `totalAmount` tính ở server từ giá sản phẩm tại thời điểm đặt.
- Mỗi dòng `quantity` từ 1 đến 20; tối đa 50 dòng mỗi đơn; không trùng `productId`.
- Idempotency: cùng `Idempotency-Key` + cùng body trong 24 giờ → trả lại đúng response lần đầu; cùng key khác body → `409 IDEMPOTENCY_CONFLICT`.
- Hủy đơn ở trạng thái `CONFIRMED` bắt buộc có `reason`; khách chỉ được hủy khi đơn còn `PENDING`.
- Yêu cầu trả hàng chỉ hợp lệ trong 7 ngày kể từ khi `DELIVERED`.

- `If-Match` lệch `version` hiện tại → `412 Precondition Failed`.

10.5 Yêu cầu tài liệu hóa

Mã	Yêu cầu	Tiêu chí nghiệm thu
D1	Tách 4 nhóm spec: `public-v1`, `public-v2`, `admin`, `partner`	Swagger UI có dropdown 4 nhóm; mỗi nhóm chỉ chứa endpoint của nó
D2	Khai báo 2 security scheme: `bearerAuth` (JWT), `partnerKey` (API key header)	Mỗi operation gắn đúng scheme; nút **Authorize** dùng được trên UI
D3	`Payment` đa hình	Schema là `oneOf` 3 kiểu, có `discriminator.propertyName = method` và `mapping`; mỗi kiểu có example
D4	Phân trang và lọc	`Pageable` bung thành `page`, `size`, `sort`; `sort` chỉ cho phép giá trị đã liệt kê; schema response phân trang ổn định (không lộ cấu trúc `PageImpl`)
D5	Header dùng chung	`Idempotency-Key` (request) và `X-Request-Id` (mọi response) thêm bằng customizer, không lặp annotation ở từng method
D6	Danh mục lỗi	Schema `ApiProblem` (ProblemDetail + `code`, `errors[]`); mỗi operation chỉ liệt kê mã lỗi nó thực sự trả; mỗi lỗi có example
D7	Example có tên	`POST /api/orders` có ít nhất 2 example request (COD, CARD) chọn được trên UI
D8	Versioning	v1 có `deprecated: true` và mô tả ngày ngừng hỗ trợ; v2 tài liệu hóa đúng schema mới
D9	Spec là artifact	Build sinh `docs/api/openapi-{group}.yaml`; test CI fail khi spec đổi mà file chưa được commit lại
D10	Chất lượng spec	Lint bằng Redocly CLI hoặc Spectral: 0 error
D11	Spec dùng được	Sinh client TypeScript bằng openapi-generator từ `public-v2`, biên dịch không lỗi
D12	Production	Swagger UI tắt; `/v3/api-docs/partner` chỉ trả về khi có `X-Partner-Key` hợp lệ

10.6 Ràng buộc kỹ thuật

Spring Boot 4.1.x, JDK 25, springdoc-openapi 3.1.x, Spring Security OAuth2 Resource Server (JWT ký HS256 cho dev), Flyway cho schema, MapStruct, test tích hợp bằng Testcontainers MySQL hoặc H2.

10.7 Sản phẩm nộp

- Repository mã nguồn + `README` hướng dẫn chạy bằng một lệnh.
- Thư mục `docs/api/` chứa 4 file spec YAML.
- Bộ test: test hợp đồng spec cho từng nhóm, test máy trạng thái (mọi cặp chuyển hợp lệ và không hợp lệ), test idempotency.
- ADR 1 trang: vì sao chọn versioning bằng header thay vì path (hoặc ngược lại).